

Building Nozy Agent: Private Infrastructure for the Next Generation of Zcash Tools

A self-hosted roadmap for wallet assistance, private commerce, node operations, research, responsible automation and long-term financial education

Artificial intelligence is quickly becoming another interface to the internet. People are beginning to ask agents to research topics, organize work, monitor systems and prepare financial actions. But adding a chat box to a wallet is not enough—and giving a language model direct control over money is not a responsible shortcut.

NozyWallet is planning a different kind of foundation: **Nozy Agent**, a self-hosted, Rust-first agent system designed around Zcash, local control and explicit trust boundaries.

That agent does not start from zero. Nozy already ships a shielded-first stack users can run today:

- **CLI** production downloads (Orchard + Ironwood)
- **Desktop** production downloads (beta tag; no formal GA label)
- **Browser extension** live on the [Chrome Web Store](#) as **NozyWallet 2.0.0 (Sweet Chili)** — keys stay on the device; if no local node is found, compact sync can use the public `lwd.nozywallet.org` path
- **Localhost companion API** for same-machine sync and shared wallet profiles

The goal is to keep the **private model** intact: shielded money, local inference, Nym-gated research, and honest warnings whenever a flow would push users onto transparent or compliance-gated rails. Seamless UX must never erase that boundary.

This article is a roadmap for the **agent layer**, not a claim that every specialist desk below already ships. Many components still require engineering, testing, security review and community feedback. Publishing the direction early matters because private financial infrastructure should be designed in public before users are asked to trust it.

The real challenge is infrastructure, not conversation

A useful wallet agent must do more than produce convincing text. It needs to know which information is authoritative, which tools it may use, when it must ask for approval and what it must never be able to access.

That creates a harder set of questions:

- How can an agent help without holding spending keys?
- How can it search the web without exposing the user's IP address and interests?
- How can it distinguish a transaction proposal from permission to spend?
- How can merchants automate orders without releasing goods before payment?
- How can node operators receive useful help without allowing arbitrary remote commands?
- How can local learning improve the experience without silently changing safety rules?
- How can a friendly virtual pet teach financial preparedness without manipulating users into depositing, trading or taking risks?

Nozy Agent is being planned around these questions from the beginning.

A local-first architecture

The proposed agent will run as a separate `nozy-agent` product instead of being embedded directly inside the existing web dashboard. Its core is planned as a Rust service using Axum and a custom supervisor that routes work to narrowly scoped specialists.

The language model will run locally through Ollama by default. Operators with suitable hardware may use a localhost vLLM deployment through the same OpenAI-compatible interface. Cloud chat APIs are not part of the default trust model.

The separation of responsibilities is deliberate:

- **The local model interprets requests and proposes work.**
- **The Nozy Agent policy layer decides which tools are permitted.**
- **The wallet companion remains responsible for wallet operations.**
- **The user approves actions that can move funds.**
- **Spending keys never enter prompts, agent memory or model logs.**

The model is treated as an untrusted proposer, not as a signer or policy authority. A persuasive answer cannot bypass deterministic Rust checks.

Nym is a boundary, not a marketing label

Local inference protects prompts from a cloud model provider, but an agent may still need external information. Market quotes, research sources and community updates can reveal a great deal about a user when fetched over a normal internet connection.

The planned Nozy Agent therefore makes Nym a requirement for non-local HTTP traffic. Local connections to the wallet companion, local model and local database remain on the machine. External research and market requests must pass through the configured Nym helper, with no clearnet fallback.

If that privacy route is unhealthy, the agent and its web surface should fail closed and explain what the operator needs to repair. The existing wallet should continue functioning independently.

This does not mean Nym makes every remote service trustworthy. A quote provider can still return bad data, a webpage can contain prompt injection and a trading venue can still have custody or settlement risks. Nym addresses network metadata exposure; validation, source provenance and policy controls remain separate requirements.

Specialists instead of one all-powerful agent

Nozy Agent is planned as a supervised collection of specialist desks. Specialists do not freely call one another or acquire new tools. The supervisor controls handoffs through typed shared state, hop limits and role-specific permissions.

The roadmap currently includes:

- **Money and Guide:** explain balances, privacy choices and wallet workflows, then prepare proposals for explicit approval.
- **Memory:** keep local notes, corrections, preferences and reusable playbooks without storing mnemonics, passwords or raw keys.
- **Market and Alerts:** collect allowlisted information through Nym, cache it locally and distinguish sourced facts from model interpretation.
- **Node Operations:** guide self-hosted Zebra and lightwalletd operators through health checks and documented maintenance.
- **Protocol Debug:** help CLI developers collect local diagnostics and reason about Zcash pool, witness, anchor and transaction failures without pretending NozyWallet is a consensus node.

- **Merchant Desk:** accept customer orders, create shielded ZEC invoices and release work only after the companion reports the required confirmation state.
- **Bounty Hunter:** maintain a private casebook, help reproduce issues, enforce structured reporting gates and support privacy-preserving public reputation based on verifiable acknowledgements.
- **Zcash Research and Knowledge:** answer questions about Zcash development, teams, history and community using a curated local corpus, the Zcash Community Forum and Nym-routed web research with citations.

This design gives every specialist less authority than a general-purpose agent. That limitation is a feature.

Private commerce needs deterministic handoffs

The Merchant Desk borrows the useful shape of modern commerce protocols: declared capabilities, checkout sessions and modular payment handlers. Nozy's default payment handler, however, is a shielded ZEC invoice—not a card processor or unrelated payment network.

A customer may be able to talk with the merchant agent, choose an item and receive an invoice without occupying a worker. But the language model cannot decide that a promise, screenshot or friendly message counts as payment.

The intended flow is deterministic:

1. Match the request to an approved catalog.
2. Create a checkout session.
3. Ask the wallet companion to create the invoice.
4. Monitor the authoritative invoice state.
5. Produce exactly one release ticket only after the configured confirmation requirement is met.

This is the type of infrastructure that turns automation into a dependable business tool.

Better tools for operators, researchers and bounty hunters

Running private financial infrastructure requires maintenance. Nozy Agent is intended to help users understand their own node stack, gather diagnostics and follow reviewed playbooks. It should explain what it observed and cite the command or probe that produced the evidence.

The same principle applies to security research. AI can help investigators organize notes and create reproductions, but it can also generate low-quality reports or accidentally publish exploitable details. The Bounty Hunter Desk is planned around private case files, target-specific scope, reproducibility checklists, evidence requirements and responsible disclosure.

Public hunter profiles may eventually show a stable pseudonymous handle and verified acknowledgements without displaying wallet addresses. Unverified claims must remain visibly separate from credits signed or published by the relevant project. Nozy would provide tooling and reputation infrastructure—not replace official Zcash ecosystem bounty programs.

The Research specialist provides another shared foundation. It will prioritize primary sources and curated Zcash resources, especially the Zcash Community Forum, while isolating web content as untrusted data. Answers should include dates and citations so users can distinguish current facts, historical context, proposals and community opinion.

A bounded trading system — and why private-to-transparent exits are dangerous

A separate trading roadmap explores support for external swap and DEX venues through reviewed adapters. The architecture is venue-neutral: each venue must define quote, execution, settlement, timeout and refund behavior instead of receiving broad wallet access.

What we know about leaving the shielded pool

NozyWallet's default product posture is **shielded-first and private by design**. Orchard (and related shielded pools) hide amounts, counterparties and much of the graph that transparent blockchains publish by default. That privacy is not a cosmetic feature. It is the difference between money that can be held and spent without building a permanent public dossier, and money that is continuously observable.

Moving value from **private (shielded ZEC)** into **transparent rails** (transparent Zcash addresses, ETH addresses, USDT deposits, cross-chain solvers, AML-screened bridges) is therefore not a small UX detail. It is a **trust-model change**:

1. **The privacy set collapses.** Once funds leave the shielded pool into a transparent deposit or settlement address, amount and timing become linkable. Even if the earlier shielded history was private, the exit creates a new public edge.
2. **A third party can intervene.** Transparent and compliance-gated rails can screen addresses, delay credit, freeze, re-open reviews and require support processes. That is not the same as Orchard's "your keys, your notes" model.
3. **Recovery often costs more privacy.** When an integrated exit fails, users may be pushed into email tickets, Telegram, public posts and identity-adjacent documentation—the opposite of why they held shielded ZEC.
4. **Marketing can lie about the boundary.** If a wallet packages a compliance-gated swap as "trustless" or "decentralised" inside the same shielded experience, users reasonably misjudge risk and size.

This is not theoretical. In September 2026, a Zcash Community Forum report described exiting shielded ZEC from Zodl through NEAR Intents, converting roughly 1,120 ZEC into about **\$589,000 USDT**, then having those funds held for about **fifty days** despite a written compliance clearance that was later reopened:

[My experience exiting shielded ZEC from Zodl to NEAR Intents: \\$589k USDT still held 50 days despite a written compliance clearance](#)

The shielded pool itself was not the failure. The integrated exit was. The post separates three layers—shielded Zcash, the wallet, and NEAR Intents—and shows how a seamless UI can hide a hard change from private money to public, holdable settlement.

What Nozy Agent will refuse to blur

We want the **private model**, not the public one, as far as honest engineering allows:

- Keep ordinary wallet activity **shielded-first**.
- Keep agent inference and memory **local**.
- Keep agent web and research egress on **Nym**, not clearnet.
- Keep merchant defaults on **shielded ZEC invoices**, not card rails.
- Treat any private→transparent or cross-chain exit as an **explicit, warned, size-limited boundary**, never as "still private because it started shielded."
- Prefer venues and flows that preserve privacy end-to-end. Where that is impossible with today's rails, say so before the user clicks.

Nym can hide network origin for quotes. It does **not** restore Orchard privacy after a transparent deposit, and it does **not** stop AML/KYT holds.

NEAR Intents demotion (locked)

NEAR Intents is demoted to paper trading and tiny human-approved canaries only. After forum [#57497](#), Nozy will not treat that rail as a safe first venue for size or automated live trading.

Required product rules:

- Never call NEAR “trustless,” “non-custodial like Orchard,” or “same privacy as shielded ZEC.”
- Show a pre-trade trust-boundary warning in the UI itself.
- Trip circuit breakers on uncredited deposits, compliance holds, clearance-then-reopen and missing refund transaction hashes.
- Require a separate review of settle/refund SLAs and reclaim paths before any promotion past canary.

Trading would use a dedicated, capped wallet process separated from the user’s main wallet. A deterministic Rust risk engine would enforce user-defined mandates, exposure limits, loss limits, stale-quote checks and circuit breakers. The model could select among permitted strategies and learn from recorded outcomes, but it could not rewrite those limits.

Continuous learning is planned for prediction calibration, market-regime detection and strategy memory—not for loosening custody boundaries after a winning streak.

Paper trading, adversarial fixtures and small canary limits must come before meaningful funds. External venues may expose transparent addresses, timing, AML/KYT holds or cross-chain metadata even when quote requests use Nym. Those trade-offs must be shown before approval, not hidden behind an animated interface.

The optional Pet: education without financial pressure

The planned Nozy Pet gives these systems a more approachable interface. It is optional, off by default and never a signer.

In **Learn & Save** mode, the pet can guide users through Zcash, Nozy, Nym, backups, privacy and private commerce. Users may set a savings target and view read-only progress, but the pet cannot lock funds, shame withdrawals, promise returns or reward larger deposits. Growth comes from completing educational lessons and demonstrating understanding.

In **Trading Pet** mode, the character becomes a visual representation of authoritative trading-engine state. It might appear calm in a flat market, cautious when liquidity is poor or sheltered when a circuit breaker is active. Every animation must have an evidence card and an accessible data-only equivalent. The pet cannot celebrate trade volume, pressure the user to recover losses or initiate a trade.

Legacy Guardian and milestone rewards

The **Legacy Guardian** concept extends Learn & Save into long-term preparedness. It draws inspiration from the lifecycle ideas in the open-source [Legacy Contract](#): an owner, trusted people, a long-term plan, periodic check-ins and maturity.

Nozy Agent v1 is not planned to deploy or connect to that EVM contract. The repository itself explains that its contract does not custody or move ZEC; it records a policy while an off-chain wallet or operator must perform the actual financial action. It is also unaudited, and publishing a Zcash note commitment alongside EVM activity could create a serious privacy link.

Instead, Legacy Guardian is planned as a local encrypted preparedness tool:

- help the founder define trusted contacts and beneficiary labels;
- schedule private plan reviews and check-ins;
- teach backup and recovery responsibilities;

- prepare an owner-reviewed offline instruction package; and
- track optional milestones connected to future gifts or rewards.

A founder could create milestones such as completing high school and then college, with a separate reward assigned to each stage. With the heir's consent, the agent could inspect evidence locally, extract the required fields, validate a signed credential or follow an official reference through Nym, check the evidence against the founder's written rules and prepare a recommendation.

But an AI model cannot determine universal truth from a screenshot. Screenshots can be edited, generated or associated with the wrong person. The verification strength must match the value and consequences of the reward:

- A low-value milestone might use documentary evidence followed by founder approval.
- A medium-value milestone might require an official reference plus a designated trustee.
- A high-value or inheritance-related milestone should require an issuer-signed credential or direct institutional verification and a predefined trustee quorum.

The founder must define acceptable evidence, deadlines, fallback rules and disputes while creating the milestone. If a requirement is ambiguous, the agent must not invent a verification method later. A missed check-in must never be treated as proof of death or incapacity.

Milestone evidence may contain sensitive educational records and information about minors. It must remain in a dedicated encrypted local store, be excluded from ordinary LLM memory and logs, and never be sent to a cloud vision or document service. The heir must be able to see what is being checked and what result will be shared with trustees.

For the first phase, the agent should only recommend that a milestone is sufficiently supported. The founder or designated trustees would approve any wallet proposal. Automatic inheritance or escrow release would be a separate, audited protocol project with its own privacy review—not a hidden power of the pet.

Local learning with immutable safety boundaries

Nozy Agent is intended to improve as users correct it and complete tasks. That does not require silently fine-tuning model weights.

The first learning layer can remain inspectable: local episodes, corrections, notes, playbooks, source preferences and outcome records stored in SQLite with local embeddings. Users should be able to inspect, forget, export or wipe this memory.

Learning may improve which explanation the Guide offers, how research is organized or which permitted trading strategy is predicted to fit a market regime. It may not grant a new tool, increase a spending cap, disable Nym or move a task from "requires approval" to "automatic."

The system should get better at helping while remaining unable to get better at bypassing the owner.

Operator-owned deployment

The default deployment remains the user's own machine or private server. The roadmap also considers first-party Akash deployment definitions for operator-controlled ZebraD/lightwalletD infrastructure and optional GPU-backed local-model serving.

Akash does not replace Nym, and a remote lease is not the same trust boundary as a computer in the user's home. Spending keys should not be placed on those deployments. Enterprise operators may choose private-access or zero-trust networking products to keep services less exposed, but those products are deployment notes rather than Nozy dependencies.

The goal is portability without pretending every deployment location offers equal privacy.

Building in phases

This roadmap needs staged delivery:

1. Publish the issue, RFC, threat model, requirements and funding evidence.
2. Build the Rust agent skeleton with local-model and Nym health gates.
3. Add policy-controlled read tools, wallet proposals and explicit approvals.
4. Deliver merchant, operator, debugging and bounty workflows with deterministic acceptance tests.
5. Add citation-backed research and local learning.
6. Introduce the optional Pet, Legacy Guardian and milestone evidence workflows without financial authority.
7. Develop trading separately through paper mode, reviewed venue adapters and hard risk limits.
8. Provide operator deployment templates, runbooks and independent security review where required.

Tests must cover the failures that matter: attempts to place mnemonics in memory, prompt injection requesting a direct send, customer sessions asking for owner data, clearnet access while Nym is unavailable, duplicate merchant release and unsupported claims of milestone completion.

Why build this now?

Infrastructure determines what future applications can safely become.

If agentic wallet products begin with unrestricted model access, remote inference and vague approvals, safety will be difficult to add later. If they begin with key isolation, typed capabilities, private networking, evidence provenance and deterministic policy, new interfaces can be added without discarding the trust model.

The same foundations can support a first-time Zcash user learning about shielded transactions, a merchant accepting private payments, an operator maintaining a node, a researcher preparing a responsible disclosure, a family documenting a long-term plan and a power user experimenting with bounded strategies.

Nozy Agent is ambitious because the opportunity is larger than a chatbot. The work is to build private, inspectable infrastructure that helps people act—while preserving the human authority to decide.

That is the future NozyWallet is planning toward.

Publication notes

Suggested social preview: Nozy already ships CLI, Desktop, and a Chrome Web Store extension. Next is Nozy Agent—local AI, Nym-required web access, shielded-first commerce, and honest trust boundaries. After a public NEAR Intents hold case (~\$589k for ~50 days), private-to-transparent exits are treated as dangerous demotions of privacy, not “still private” swaps.

Suggested Medium tags: Zcash, Privacy, Artificial Intelligence, Open Source, Fintech

Project links:

- NozyWallet repository: <https://github.com/LEONINE-DAO/Nozy-wallet>
- Chrome Web Store (2.0.0 Sweet Chili):
<https://chromewebstore.google.com/detail/nozywallet/kjbpoimgafbikmachliikhoamdlnhooj>
- Enhancement roadmap: https://github.com/LEONINE-DAO/Nozy-wallet/blob/master/ENHANCEMENT_ROADMAP.md
- This draft (Markdown): docs/medium/BUILDING_NOZY_AGENT_INFRASTRUCTURE.md
- This draft (PDF): docs/medium/BUILDING_NOZY_AGENT_INFRASTRUCTURE.pdf

- Cited forum case: <https://forum.zcashcommunity.com/t/my-experience-exiting-shielded-zec-from-zodl-to-near-intents-589k-usdt-still-held-50-days-despite-a-written-compliance-clearance/57497>
- Nozy Agent RFC/issue: [ADD AFTER OPENING]

Status disclosure: CLI, Desktop downloads, and the Chrome extension are shipping surfaces. The agent specialists and trading/pet features below are planned work and may change after technical investigation, security review and community feedback. The NEAR Intents demotion reflects publicly reported settlement/hold risk and Nozy's private-by-default product stance; it is not a legal finding about any party.

AI-assistance disclosure: AI tools assisted with organizing and editing this draft. The NozyWallet maintainers are responsible for validating its technical claims, product decisions and final published text.